

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №3

по дисциплине «Численные методы решения прикладных задач»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.х.н., доцент (ОНД, ИШПР)

_____ Зарубин А.Г.

Цель

Изучить и проработать базовое решение (baseline) задачи прогнозирования спроса из соревнования Sibur Challenge 2021, а также предложить более точное решение на основе CatBoostRegressor.

Задание

1. Ознакомиться с условием задачи прогнозирования спроса из хакатона Sibur Challenge 2021.
2. Загрузить датасет `sc2021_train_deals.csv` и преобразовать его в панель временных рядов.
3. Воспроизвести baseline-решение организаторов: наивные модели (последнее значение, среднее за 3 месяца) и CatBoostRegressor с набором признаков из оригинального ноутбука.
4. Зафиксировать метрики качества (RMSLE) всех трёх baseline-моделей на кросс-валидации TimeSeriesSplit.
5. Предложить более точное решение на основе CatBoostRegressor с расширением признакового пространства и корректной функцией потерь.
6. Сравнить полученные результаты с baseline и сделать выводы.

Выполнение работы

1. Описание baseline-решения

Для переиспользования между файлами вынесены общие части: пути, список идентификаторов групп, диапазон целевых месяцев, реализация RMSLE и загрузчик данных. Код приведён на рисунке 1.

```
1  import pathlib
2  import numpy as np
3  import pandas as pd
4
5  DATA_DIR = pathlib.Path(__file__).resolve().parent
6  DATA_FILE = "sc2021_train_deals.csv"
7
8  AGG_COLS = ["material_code", "company_code", "country", "region", "manager_code"]
9  GROUPS = 941
10
11  TARGET_FULL_RANGE = pd.date_range("2019-01-01", "2020-07-01", freq="MS")
12  RS = 82736
13
14
15  def rmsle(y_true, y_pred):
16      """RMSLE с обрезкой отрицательных прогнозов."""
17      y_pred = np.clip(np.asarray(y_pred, dtype=float), 0.0, None)
18      y_true = np.asarray(y_true, dtype=float)
19      return float(np.sqrt(np.mean((np.log1p(y_true) - np.log1p(y_pred)) ** 2)))
20
21
22  def load_group_ts(data_dir=DATA_DIR):
23      """Загружает данные и строит матрицу group x month."""
24      data_dir = pathlib.Path(data_dir)
25      data = pd.read_csv(data_dir / DATA_FILE, parse_dates=["month", "date"])
26
27      group_ts = (
28          data.groupby(AGG_COLS + ["month"])["volume"]
29          .sum()
30          .unstack(fill_value=0)
31          .sort_index(axis=1)
32      )
33      return group_ts
34
35
36  def month_value(group_ts, month):
37      """Значение месяца или нули, если месяца нет в данных."""
38      if month in group_ts.columns:
39          return group_ts[month].astype(float)
40      return pd.Series(0.0, index=group_ts.index)
```

Рисунок 1. Содержимое common.py

Ключевые функции:

- `rmsle(y_true, y_pred)` — своя реализация метрики с обрезкой отрицательных прогнозов через `np.clip(..., 0, None)`. Отсутствует зависимость от устаревшей `sklearn.metrics.mean_squared_log_error`.
- `load_group_ts()` — читает CSV и строит матрицу 941×31 . Группируем по (группа, месяц), суммируем `volume`, `unstack` выносит месяцы в столбцы, `fill_value=0` превращает «нет сделок» в 0, `sort_index(axis=1)` гарантирует хронологический порядок столбцов.
- `month_value(group_ts, month)` — безопасное получение колонки месяца (если месяца нет, возвращает Series из нулей вместо `KeyError`)

Для оценки примитивных baseline-моделей реализована функция `evaluate_simple`, код приведён на рисунке 2.

```

28 # Общая функция оценки наивных baseline
29
30 def evaluate_simple(features, feature_col, name):
31     print(f"\n=== {name} ===")
32
33     ts_cv = TimeSeriesSplit(n_splits=5, test_size=GROUPS * 3)
34     losses = []
35
36     for fold, (train_idx, test_idx) in enumerate(ts_cv.split(features), 1):
37         train = features.iloc[train_idx]
38         test = features.iloc[test_idx]
39
40         y_train = train["target"].values
41         y_test = test["target"].values
42
43         train_loss = rmsle(y_train, train[feature_col].values)
44         test_loss = rmsle(y_test, test[feature_col].values)
45
46         losses.append([train_loss, test_loss])
47
48         print(f"Fold {fold}")
49         print(
50             f"  Train: {train['target_month'].min().date()} - "
51             f"{train['target_month'].max().date()} | RMSLE={train_loss:.6f}"
52         )
53         print(
54             f"  Test:  {test['target_month'].min().date()} - "
55             f"{test['target_month'].max().date()} | RMSLE={test_loss:.6f}"
56         )
57
58     avg = np.mean(losses, axis=0)
59     print(f"Avg Train RMSLE: {avg[0]:.6f}, Avg Test RMSLE: {avg[1]:.6f}")
60

```

Рисунок 2. Функция `evaluate_simple`.

Функция принимает датафрейм признаков и имя колонки-прогноза. Внутри:

- Создается `TimeSeriesSplit(n_splits=5, test_size=GROUPS*3)`.
- Для каждого фолда вырезаются train/test, «прогнозом» считается значение колонки `feature_col` — обучения не происходит.
- Считается RMSLE на train и test.
- Печатаются диапазоны дат фолдов и метрики.
- Это единый бенчмарк, позволяющий сравнивать все модели в одинаковых условиях.

Baseline #1 — последнее известное значение

Идея: прогноз объёма на следующий месяц равен объёму текущего месяца. Функция `make_last_value_features` формирует для каждого из 19 целевых месяцев 941 строку с признаком `vol_tm1` и целевой переменной `target`. Код приведён на рисунке 3.

```

61 # Baseline #1: last value
62
63 def make_last_value_features(group_ts):
64     rows = []
65
66     for target_month in TARGET_FULL_RANGE:
67         prev_month = target_month - pd.offsets.MonthBegin(1)
68
69         df = pd.DataFrame(index=group_ts.index)
70         df["vol_tm1"] = month_value(group_ts, prev_month)
71         df["target"] = month_value(group_ts, target_month)
72         df["target_month"] = target_month
73
74         rows.append(df.reset_index())
75
76     features = pd.concat(rows, ignore_index=True)
77     features = features.sort_values("target_month", kind="mergesort").reset_index(drop=True)
78
79     assert features["target_month"].is_monotonic_increasing
80     assert (features["target_month"].value_counts() == GROUPS).all()
81
82     return features
83

```

Рисунок 3. Функция `make_last_value_features`.

Внутри каждой итерации — три колонки: значение прошлого месяца, значение целевого месяца и сам `target_month`. После конкатенации выполняются две проверки: `target_month` монотонно возрастает, и в каждом месяце ровно 941 группа. Результат кросс-валидации приведён на рисунке 4.

```

=== Baseline #1: last value ===
Fold 1
  Train: 2019-01-01 - 2019-04-01 | RMSLE=1.792478
  Test:  2019-05-01 - 2019-07-01 | RMSLE=1.757021
Fold 2
  Train: 2019-01-01 - 2019-07-01 | RMSLE=1.777368
  Test:  2019-08-01 - 2019-10-01 | RMSLE=1.737792
Fold 3
  Train: 2019-01-01 - 2019-10-01 | RMSLE=1.765589
  Test:  2019-11-01 - 2020-01-01 | RMSLE=1.963820
Fold 4
  Train: 2019-01-01 - 2020-01-01 | RMSLE=1.813259
  Test:  2020-02-01 - 2020-04-01 | RMSLE=2.046455
Fold 5
  Train: 2019-01-01 - 2020-04-01 | RMSLE=1.859212
  Test:  2020-05-01 - 2020-07-01 | RMSLE=2.027146
Avg Train RMSLE: 1.801581, Avg Test RMSLE: 1.906447

```

Рисунок 4. Вывод Baseline #1.

Результат: Avg Train RMSLE = 1.8016, Avg Test RMSLE = 1.9064.

Уже на первом фолде видно, что Test RMSLE слегка ниже Train (1.757 против 1.792) — данные инерционные. Но начиная с 3-го фолда тест резко ухудшается (1.96 -> 2.05 -> 2.03). Это первые месяцы 2020 года — ковидный период, который модель не могла предсказать.

Baseline #2 — среднее за последние 3 месяца

Идея: сгладить шум одного месяца, усреднив три предыдущих. Код `make_last_3_avg_features` приведён на рисунке 5.

```

85 # Baseline #2: last 3 months average
86
87 def make_last_3_avg_features(group_ts):
88     rows = []
89
90     for target_month in TARGET_FULL_RANGE:
91         start = target_month - pd.offsets.MonthBegin(3)
92         end = target_month - pd.offsets.MonthBegin(1)
93
94         cols = pd.date_range(start, end, freq="MS")
95         cols = [c for c in cols if c in group_ts.columns]
96
97         if cols:
98             avg = group_ts.loc[:, cols].mean(axis=1)
99         else:
100             avg = pd.Series(0.0, index=group_ts.index)
101
102         df = pd.DataFrame(index=group_ts.index)
103         df["last_3m_avg"] = avg
104         df["target"] = month_value(group_ts, target_month)
105         df["target_month"] = target_month
106
107         rows.append(df.reset_index())
108
109     features = pd.concat(rows, ignore_index=True)
110     features = features.sort_values("target_month", kind="mergesort").reset_index(drop=True)
111
112     assert features["target_month"].is_monotonic_increasing
113     assert (features["target_month"].value_counts() == GROUPS).all()
114
115     return features
116

```

Рисунок 5. Функция make_last_3_avg_features

Для каждого целевого месяца формируется столбец last_3m_avg — среднее трёх предыдущих. Здесь добавлена защита: если какие-то месяцы отсутствуют в данных, они исключаются из среднего (if c in group_ts.columns). Результат — на рисунке 6.

```

=== Baseline #2: last 3 months average ===
Fold 1
  Train: 2019-01-01 - 2019-04-01 | RMSLE=1.705125
  Test:  2019-05-01 - 2019-07-01 | RMSLE=1.652521
Fold 2
  Train: 2019-01-01 - 2019-07-01 | RMSLE=1.682782
  Test:  2019-08-01 - 2019-10-01 | RMSLE=1.681824
Fold 3
  Train: 2019-01-01 - 2019-10-01 | RMSLE=1.682494
  Test:  2019-11-01 - 2020-01-01 | RMSLE=1.916876
Fold 4
  Train: 2019-01-01 - 2020-01-01 | RMSLE=1.739388
  Test:  2020-02-01 - 2020-04-01 | RMSLE=1.917399
Fold 5
  Train: 2019-01-01 - 2020-04-01 | RMSLE=1.774126
  Test:  2020-05-01 - 2020-07-01 | RMSLE=1.892962
Avg Train RMSLE: 1.716783, Avg Test RMSLE: 1.812316

```

Рисунок 6. Вывод Baseline #2.

Результат: Avg Train RMSLE = 1.7168, Avg Test RMSLE = 1.8123.

Улучшение относительно Baseline #1:

на train: -4.7%;

на test: -4.9%.

Сглаживание выигрывает на короткой истории (фолды 1–2: Test 1.65 и 1.68 против 1.76 и 1.74 у #1), но так же сильно проседает на ковидных месяцах (фолды 4–5: Test 1.92 и 1.89).

Baseline #3 — CatBoostRegressor

Первая модель с обучением. Признаки: категориальные (material_code, company_code, country, region, manager_code, month), шесть лагов vol_tm6 ... vol_tm1 и скользящие статистики за 12 месяцев: last_year_avg, last_year_min, last_year_max. Код get_features и make_catboost_features — на рисунке 7.

```
110
117 # Baseline #3: CatBoostRegressor
118
119 def get_features(group_ts: pd.DataFrame, month: pd.Timestamp) -> pd.DataFrame:
120     """Признаки для месяца `month`. В точности как в ноутбуке."""
121     start_period = month - pd.offsets.MonthBegin(6)
122     end_period = month - pd.offsets.MonthBegin(1)
123
124     df = group_ts.loc[:, :end_period]
125
126     features = pd.DataFrame([], index=df.index)
127     features["month"] = month.month
128     features[[f"vol_tm{i}" for i in range(6, 0, -1)]] = df.loc[:, start_period:end_period].copy()
129
130     roll_t = df.T.rolling(window=12, min_periods=1)
131     features = features.join(roll_t.mean().iloc[-1].rename("last_year_avg"))
132     features = features.join(roll_t.min().iloc[-1].rename("last_year_min"))
133     features = features.join(roll_t.max().iloc[-1].rename("last_year_max"))
134     return features
135
136
137 def make_catboost_features(group_ts):
138     datasets = []
139     for target_month in TARGET_FULL_RANGE:
140         features = get_features(group_ts, target_month)
141         features["target"] = month_value(group_ts, target_month)
142         features["target_month"] = target_month
143         datasets.append(features.reset_index())
144
145     df = pd.concat(datasets, ignore_index=True)
146     df = df.sort_values("target_month", kind="mergesort").reset_index(drop=True)
147
148     assert df["target_month"].is_monotonic_increasing
149     assert (df["target_month"].value_counts() == GROUPS).all()
150
151     return df
```

Рисунок 7. Функции make_catboost_features и get_features.

Функция get_features

Для одного целевого месяца строит таблицу признаков — по строке на каждую из 941 группы.

Сначала определяются границы доступной истории. Для целевого месяца берётся конец предыдущего месяца (это последний месяц, данные которого можно использовать — иначе утечка из будущего) и начало окна «шесть

месяцев назад». Всё, что правее конца предыдущего месяца, отрезается от матрицы `group_ts`.

Затем для каждой группы формируется набор признаков:

- Календарный признак — номер месяца (например, 3 для марта). Нужен, чтобы модель могла уловить сезонность: в разные месяцы спрос ведёт себя по-разному.
- Шесть лагов — значения за шесть месяцев до целевого. Самый свежий — месяц непосредственно перед целевым, самый старый — шесть месяцев назад. То есть признаки называются `vol_tm1` (лаг 1) ... `vol_tm6` (лаг 6). Эти значения берутся из заранее отрезанной истории, поэтому в признаки не попадает ничего из будущего.
- Три скользящие статистики за 12 месяцев — среднее, минимум и максимум по последнему году. Проблема в том, что `rolling` в `pandas` скользит по строкам, а нам нужно по времени. Поэтому матрица транспонируется: теперь строки — месяцы, столбцы — группы. Скользящее окно шириной 12 месяцев двигается по времени, а параметр `min_periods=1` позволяет считать статистику даже когда истории меньше года. Затем берётся последняя строка — значения на конец доступной истории — и она присоединяется к таблице признаков как три отдельные колонки.

Итог: таблица с 941 строкой и десятью колонками — номер месяца, шесть лагов и три годовые статистики. Целевая переменная здесь ещё не добавлена.

Функция `make_catboost_features`

Собирает признаки для всех 19 целевых месяцев сразу.

Для каждого месяца вызывается `get_features`, затем к таблице добавляются две колонки: целевая переменная (значение целевого месяца) и служебная колонка `target_month` с самим месяцем. Последняя нужна для кросс-валидации и проверок. После этого индекс из пяти идентификаторов группы разворачивается в обычные колонки — таблица становится плоской, одна строка = одна пара (группа, месяц).

Все 19 таблиц склеиваются в один датасет из 17879 строк (941×19). Затем строки сортируются по `target_month` — это обязательное требование `TimeSeriesSplit`. Сортировка устойчивая, чтобы порядок групп внутри месяца не менялся.

В конце — две проверки. Первая: `target_month` монотонно возрастает, иначе кросс-валидация разобьёт строки неправильно. Вторая: в каждом месяце ровно 941 строка — если меньше, значит где-то потерялась группа.

Оценка — в `evaluate_catboost_baseline`: на каждом фолде создаётся `CatBoostRegressor` (`iterations=1000`, `depth=6`, `early_stopping_rounds=30`) и обучается на сыром `target`. Код и результат — на рисунках 8 и 9.

```
153 def evaluate_catboost_baseline(df, name):
154     print(f"\n=== {name} ===")
155
156     base_model = CatBoostRegressor(
157         iterations=1000,
158         early_stopping_rounds=30,
159         depth=6,
160         cat_features=CAT_COLS,
161         random_state=RS,
162         verbose=False,
163         allow_writing_files=False,
164     )
165
166     ts_cv = TimeSeriesSplit(n_splits=5, test_size=GROUPS * 3)
167     losses = []
168
169     for fold, (train_idx, test_idx) in enumerate(ts_cv.split(df), 1):
170         X_train = df[FTS_COLS].iloc[train_idx]
171         X_test = df[FTS_COLS].iloc[test_idx]
172         y_train = df[TARGET].iloc[train_idx]
173         y_test = df[TARGET].iloc[test_idx]
174
175         model = base_model.copy()
176         model.fit(X_train, y_train, eval_set=(X_test, y_test))
177
178         train_loss = rmsle(y_train, model.predict(X_train))
179         test_loss = rmsle(y_test, model.predict(X_test))
180
181         losses.append([train_loss, test_loss])
182
183         print(f"Fold {fold}")
184         print(
185             f"  Train: {df['target_month'].iloc[train_idx].min().date()} - "
186             f"{df['target_month'].iloc[train_idx].max().date()} | RMSLE={train_loss:.6f}"
187         )
188         print(
189             f"  Test: {df['target_month'].iloc[test_idx].min().date()} - "
190             f"{df['target_month'].iloc[test_idx].max().date()} | RMSLE={test_loss:.6f}"
191         )
192
193     avg = np.mean(losses, axis=0)
194     print(f"Avg Train RMSLE: {avg[0]:.6f}, Avg Test RMSLE: {avg[1]:.6f}")
195
196
```

Рисунок 8. Функция `evaluate_catboost_baseline`.

```

=== Baseline #3: CatBoostRegressor ===
Fold 1
  Train: 2019-01-01 - 2019-04-01 | RMSLE=1.932906
  Test:  2019-05-01 - 2019-07-01 | RMSLE=1.947802
Fold 2
  Train: 2019-01-01 - 2019-07-01 | RMSLE=2.053931
  Test:  2019-08-01 - 2019-10-01 | RMSLE=1.908815
Fold 3
  Train: 2019-01-01 - 2019-10-01 | RMSLE=2.226333
  Test:  2019-11-01 - 2020-01-01 | RMSLE=2.250038
Fold 4
  Train: 2019-01-01 - 2020-01-01 | RMSLE=2.171322
  Test:  2020-02-01 - 2020-04-01 | RMSLE=2.211588
Fold 5
  Train: 2019-01-01 - 2020-04-01 | RMSLE=2.078048
  Test:  2020-05-01 - 2020-07-01 | RMSLE=2.221429
Avg Train RMSLE: 2.092508, Avg Test RMSLE: 2.107934

```

Рисунок 9. Вывод Baseline #3

Результат: Avg Train RMSLE = 2.0925, Avg Test RMSLE = 2.1079.

2. Вывод по baseline

Baseline #1 (последнее значение). Avg Test RMSLE = 1.9064. Простейшая эвристика: прогноз = объём прошлого месяца. Работает за счёт инерционности данных, но сильно проваливается на ковидных месяцах (фолды 3–5: 1.96 -> 2.05 -> 2.03).

Baseline #2 (среднее за 3 месяца). Avg Test RMSLE = 1.8123. Сглаживание убирает шум одного месяца и даёт -4.9% к Baseline #1. Лучший из наивных бейзлайнов. На ковидных месяцах всё так же проседает (1.92, 1.89).

Baseline #3 (CatBoost). Avg Test RMSLE = 2.1079. Первая модель с обучением — и она проиграла обоим наивным бейзлайнам. Причины: обучение на сыром target при логарифмической метрике RMSLE, всего 6 лагов и 3 годовые статистики, нет трендов, разностей и календарных признаков, грубая ранняя остановка. Именно этот baseline — точка отсчёта для улучшения.

3. Улучшенное решение — CatBoostRegressor

В отличие от Baseline #3, где CatBoost обучался на сыром target с шестью лагами и тремя годовыми статистиками, в улучшенном решении исправлены

три ключевых момента: функция потерь приведена в соответствие с метрикой, признаковое пространство расширено, а параметры регуляризации подобраны аккуратнее. Код функций `get_features_for_month`, `build_dataset` и `run_cv` приведён на рисунках 12 и 13.

```
20 def get_features_for_month(group_ts, target_month):
21     """Признаки для одного target_month. Используется только история до target_month."""
22     end = target_month - pd.offsets.MonthBegin(1)
23     hist = group_ts.loc[:, :end]
24
25     if hist.shape[1] == 0:
26         hist = pd.DataFrame(0.0, index=group_ts.index, columns=[end])
27
28     features = pd.DataFrame(index=group_ts.index)
29
30     # Календарные признаки
31     features["month"] = target_month.month
32     features["quarter"] = target_month.quarter
33     features["year"] = target_month.year
34     features["month_idx"] = (target_month.year - 2018) * 12 + target_month.month
35     features["month_sin"] = np.sin(2 * np.pi * target_month.month / 12)
36     features["month_cos"] = np.cos(2 * np.pi * target_month.month / 12)
37
38     # Лаги 1..12
39     for lag in range(1, 13):
40         m = target_month - pd.offsets.MonthBegin(lag)
41         features[f"vol_tm{lag}"] = month_value(group_ts, m)
42
43     # Скользящие статистики (по времени, т.е. по столбцам hist)
44     # Транспонируем, чтобы rolling работал вдоль оси времени
45     hist_t = hist.T # shape: (месяцы, группы)
46     for w in (3, 6, 12):
47         roll_t = hist_t.rolling(window=w, min_periods=1)
48         features[f"mean_{w}"] = roll_t.mean().iloc[-1]
49         features[f"std_{w}"] = roll_t.std().iloc[-1].fillna(0.0)
50         features[f"min_{w}"] = roll_t.min().iloc[-1]
51         features[f"max_{w}"] = roll_t.max().iloc[-1]
52
53     # Статистики по всей истории
54     features["hist_mean"] = hist.mean(axis=1)
55     features["hist_std"] = hist.std(axis=1).fillna(0.0)
56     features["hist_min"] = hist.min(axis=1)
57     features["hist_max"] = hist.max(axis=1)
58     features["hist_nonzero_ratio"] = (hist > 0).mean(axis=1)
59
60     # Тренды и отношения
61     features["diff_1_2"] = features["vol_tm1"] - features["vol_tm2"]
62     features["ratio_1_2"] = features["vol_tm1"] / (features["vol_tm2"] + 1.0)
63     features["diff_1_3"] = features["vol_tm1"] - features["vol_tm3"]
64     features["diff_1_6"] = features["vol_tm1"] - features["mean_6"]
65     features["diff_1_12"] = features["vol_tm1"] - features["mean_12"]
66
67     features = features.fillna(0.0)
68
69     # Целевая переменная
70     features[TARGET] = month_value(group_ts, target_month)
71
72     return features
```

Рисунок 12. Функция `get_features_for_month`

```

76
77 def build_dataset(group_ts):
78     rows = []
79
80     for target_month in TARGET_FULL_RANGE:
81         df = get_features_for_month(group_ts, target_month).reset_index()
82         df["target_month"] = target_month
83         rows.append(df)
84
85     df = pd.concat(rows, ignore_index=True)
86     df = df.sort_values("target_month", kind="mergesort").reset_index(drop=True)
87
88     assert df["target_month"].is_monotonic_increasing
89     assert (df["target_month"].value_counts() == GROUPS).all()
90
91     return df
92
93
94 def run_cv(df):
95     exclude = {"target", "target_month"}
96     feature_cols = [c for c in df.columns if c not in exclude]
97
98     ts_cv = TimeSeriesSplit(n_splits=5, test_size=GROUPS * 3)
99     losses = []
100
101     for fold, (train_idx, test_idx) in enumerate(ts_cv.split(df), 1):
102         train = df.iloc[train_idx]
103         test = df.iloc[test_idx]
104
105         X_train = train[feature_cols]
106         X_test = test[feature_cols]
107
108         y_train = train[TARGET].values
109         y_test = test[TARGET].values
110
111         model = CatBoostRegressor(
112             iterations=3000,
113             learning_rate=0.03,
114             depth=6,
115             l2_leaf_reg=3.0,
116             loss_function="RMSE",
117             eval_metric="RMSE",
118             random_seed=RS,
119             od_type="Iter",
120             od_wait=100,
121             verbose=False,
122             cat_features=CAT_COLS,
123             allow_writing_files=False,
124         )
125
126         # Важно: учим на loglp(target), т.к. RMSLE = RMSE на loglp
127         model.fit(
128             X_train,
129             np.loglp(y_train),
130             eval_set=(X_test, np.loglp(y_test)),
131             use_best_model=True,
132         )
133
134         train_pred = np.expml(model.predict(X_train))
135         test_pred = np.expml(model.predict(X_test))
136
137         train_loss = rmsle(y_train, train_pred)
138         test_loss = rmsle(y_test, test_pred)
139
140         losses.append([train_loss, test_loss])
141
142         print(f"Fold {fold}")
143         print(
144             f" Train: {train['target_month'].min().date()} - "
145             f"{train['target_month'].max().date()} | RMSLE={train_loss:.6f}"
146         )
147         print(
148             f" Test: {test['target_month'].min().date()} - "
149             f"{test['target_month'].max().date()} | RMSLE={test_loss:.6f}"
150         )
151
152     avg = np.mean(losses, axis=0)
153     print(f"\nAvg Train RMSLE: {avg[0]:.6f}, Avg Test RMSLE: {avg[1]:.6f}")
154

```

Рисунок 13. Функции build_dataset и run_cv

Функция get_features_for_month

Строит признаки для одного целевого месяца по той же логике, что и в baseline: сначала отрезается история до предыдущего месяца, затем для каждой из 941 группы формируется набор признаков. Отличия — в составе признаков.

Календарные признаки. Вместо одного номера месяца добавлены пять: номер месяца, квартал, год, сквозной номер месяца от начала истории (`month_idx`) и две тригонометрические компоненты ($\sin/\cos$ от номера месяца). Последние две позволяют модели корректно воспринимать цикличность: декабрь и январь оказываются «рядом», а не на противоположных концах числовой оси.

Лаги. Вместо шести — двенадцать (`vol_tm1 ... vol_tm12`). Это даёт модели полную годовую историю ряда и позволяет улавливать годовые паттерны.

Скользящие статистики. Вместо одного окна в 12 месяцев — три окна (3, 6 и 12) и четыре статистики в каждом: среднее, стандартное отклонение, минимум и максимум. Стандартное отклонение особенно важно: оно даёт модели представление о волатильности ряда, чего в `baseline` не было вовсе. Технически всё считается через транспонирование матрицы, как и раньше, поскольку `rolling` в `pandas` работает по строкам.

Статистики по всей истории. Дополнительно считаются среднее, стандартное отклонение, минимум, максимум и доля ненулевых месяцев по всей доступной истории группы. Доля ненулевых частично компенсирует проблему разреженности данных: модель понимает, что одни группы продают стабильно, а другие — от случая к случаю.

Тренды и отношения. Добавлены разности и отношения лагов: `vol_tm1 - vol_tm2`, `vol_tm1 - vol_tm3`, `vol_tm1 - mean_6`, `vol_tm1 - mean_12`, а также отношение $\text{vol_tm1} / (\text{vol_tm2} + 1)$. Эти признаки дают модели информацию о направлении движения ряда.

Итог: 47 колонок на выходе вместо 13 в `baseline`.

Функция `build_dataset`

Работает аналогично `baseline`: для каждого из 19 целевых месяцев строится таблица, добавляется служебная колонка `target_month`, всё склеивается в один датасет из 17879 строк. Сохраняются обе проверки: монотонность `target_month` и ровно 941 строка в каждом месяце.

Функция `run_cv`

Ключевое изменение — обучение на $\log_{1p}(\text{target})$. Метрика RMSLE — это, по сути, RMSE на логарифмах. Поэтому обучать модель на логарифме целевой переменной и возвращать прогноз через `expm1` — значит оптимизировать именно то, что измеряет метрика. В baseline #3 модель училась на сырых значениях, из-за чего большие объёмы доминировали в функции потерь и качество на малых объёмах страдало.

Параметры модели также подобраны аккуратнее: `learning_rate=0.03` вместо дефолтного, `od_wait=100` вместо 30 — ранняя остановка срабатывает реже и модель успевает доучиться. `l2_leaf_reg=3.0` даёт базовую регуляризацию. Кросс-валидация — та же, что и у baseline: `TimeSeriesSplit` на 5 фолдов по 3 месяца. Для каждого фолда модель обучается заново, прогнозы возвращаются на исходную шкалу через `expm1`, после чего считается RMSLE. Запуск скрипта приведён на рисунке 14.

```
Group time series shape: (941, 31)
CatBoost dataset shape: (17879, 47)
Fold 1
  Train: 2019-01-01 - 2019-04-01 | RMSLE=1.336759
  Test:  2019-05-01 - 2019-07-01 | RMSLE=1.516423
Fold 2
  Train: 2019-01-01 - 2019-07-01 | RMSLE=1.403851
  Test:  2019-08-01 - 2019-10-01 | RMSLE=1.474754
Fold 3
  Train: 2019-01-01 - 2019-10-01 | RMSLE=1.452919
  Test:  2019-11-01 - 2020-01-01 | RMSLE=1.653161
Fold 4
  Train: 2019-01-01 - 2020-01-01 | RMSLE=1.501388
  Test:  2020-02-01 - 2020-04-01 | RMSLE=1.744810
Fold 5
  Train: 2019-01-01 - 2020-04-01 | RMSLE=1.519319
  Test:  2020-05-01 - 2020-07-01 | RMSLE=1.698597

Avg Train RMSLE: 1.442847, Avg Test RMSLE: 1.617549
```

Рисунок 14. Вывод улучшенного `CatBoostRegressor`

Улучшение относительно Baseline #3 (Avg Test RMSLE = 2.1079) — 23.3%. Относительно лучшего наивного Baseline #2 (1.8123) — 10.7%. Test RMSLE стабильно ниже Train на всех фолдах, что говорит об отсутствии

переобучения. Пик ошибки приходится на фолды 3–4 — тот же ковидный период, что и у baseline, но просадка заметно мягче: 1.65 и 1.74 против 2.25 и 2.21 у Baseline #3.

Вывод

В ходе лабораторной работы было проработано baseline-решение задачи прогнозирования спроса Sibur Challenge 2021 и предложено более точное решение на основе CatBoostRegressor.

Воспроизведены три baseline-решения. Наивная модель «последнее значение» дала Avg Test RMSLE = 1.9064, «среднее за 3 месяца» — 1.8123, CatBoost из оригинального ноутбука — 2.1079.

Выявлены ключевые недостатки Baseline #3: обучение на сыром target при логарифмической метрике RMSLE, ограниченный набор признаков (6 лагов и 3 годовые статистики), отсутствие трендов, разностей и календарных признаков, грубая ранняя остановка.

Предложено улучшенное решение на основе CatBoostRegressor со следующими изменениями:

- обучение на $\log_{1p}(\text{target})$ — прямое соответствие метрике;
- расширенное признаковое пространство: 12 лагов вместо 6, скользящие статистики (mean/std/min/max) за 3/6/12 месяцев, статистики по всей истории, тренды и отношения лагов, расширенные календарные признаки;
- более мягкие параметры обучения (learning_rate=0.03, od_wait=100, l2_leaf_reg=3.0).

Итоговое улучшение. Улучшенная модель достигла Avg Test RMSLE = 1.6175, что на 23.3% лучше Baseline #3 (2.1079) и на 10.7% лучше лучшего наивного Baseline #2 (1.8123). Test RMSLE стабильно ниже Train на всех фолдах — признак отсутствия переобучения. На ковидных месяцах (фолды 3–4) улучшенная модель показывает заметно более мягкую просадку, чем baseline: 1.65 и 1.74 против 2.25 и 2.21.

Цель работы достигнута: изучено baseline-решение, освоены методы работы с панельными временными рядами и TimeSeriesSplit, предложено решение, превосходящее все три baseline по целевой метрике.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

common.py

```
import pathlib
import numpy as np
import pandas as pd
```

```
DATA_DIR = pathlib.Path(__file__).resolve().parent
DATA_FILE = "sc2021_train_deals.csv"
```

```
AGG_COLS = ["material_code", "company_code", "country", "region", "manager_code"]
GROUPS = 941
```

```
TARGET_FULL_RANGE = pd.date_range("2019-01-01", "2020-07-01", freq="MS")
RS = 82736
```

```
def rmsle(y_true, y_pred):
    """RMSLE с обрезкой отрицательных прогнозов."""
    y_pred = np.clip(np.asarray(y_pred, dtype=float), 0.0, None)
    y_true = np.asarray(y_true, dtype=float)
    return float(np.sqrt(np.mean((np.log1p(y_true) - np.log1p(y_pred)) ** 2)))
```

```
def load_group_ts(data_dir=DATA_DIR):
    """Загружает данные и строит матрицу group x month."""
    data_dir = pathlib.Path(data_dir)
    data = pd.read_csv(data_dir / DATA_FILE, parse_dates=["month", "date"])

    group_ts = (
        data.groupby(AGG_COLS + ["month"])["volume"]
        .sum()
        .unstack(fill_value=0)
        .sort_index(axis=1)
    )
    return group_ts
```

```
def month_value(group_ts, month):
    """Значение месяца или нули, если месяца нет в данных."""
    if month in group_ts.columns:
        return group_ts[month].astype(float)
    return pd.Series(0.0, index=group_ts.index)
```

baseline.py

```
import numpy as np
import pandas as pd
from sklearn.model_selection import TimeSeriesSplit
from catboost import CatBoostRegressor
```

```
from common import (
    AGG_COLS,
    GROUPS,
    TARGET_FULL_RANGE,
    RS,
```

```

rmsle,
load_group_ts,
month_value,
)

CAT_COLS = AGG_COLS + ["month"]
TARGET = "target"

FTS_COLS = [
    "material_code", "company_code", "country", "region", "manager_code", "month",
    "vol_tm6", "vol_tm5", "vol_tm4", "vol_tm3", "vol_tm2", "vol_tm1",
    "last_year_avg", "last_year_min", "last_year_max",
]

# Общая функция оценки наивных baseline

def evaluate_simple(features, feature_col, name):
    print(f"\n=== {name} ===")

    ts_cv = TimeSeriesSplit(n_splits=5, test_size=GROUPS * 3)
    losses = []

    for fold, (train_idx, test_idx) in enumerate(ts_cv.split(features), 1):
        train = features.iloc[train_idx]
        test = features.iloc[test_idx]

        y_train = train["target"].values
        y_test = test["target"].values

        train_loss = rmsle(y_train, train[feature_col].values)
        test_loss = rmsle(y_test, test[feature_col].values)

        losses.append([train_loss, test_loss])

    print(f"Fold {fold}")
    print(
        f" Train: {train['target_month'].min().date()} - "
        f"{train['target_month'].max().date()} | RMSLE={train_loss:.6f}"
    )
    print(
        f" Test: {test['target_month'].min().date()} - "
        f"{test['target_month'].max().date()} | RMSLE={test_loss:.6f}"
    )

    avg = np.mean(losses, axis=0)
    print(f"Avg Train RMSLE: {avg[0]:.6f}, Avg Test RMSLE: {avg[1]:.6f}")

# Baseline #1: last value

def make_last_value_features(group_ts):
    rows = []

    for target_month in TARGET_FULL_RANGE:
        prev_month = target_month - pd.offsets.MonthBegin(1)

        df = pd.DataFrame(index=group_ts.index)
        df["vol_tm1"] = month_value(group_ts, prev_month)
        df["target"] = month_value(group_ts, target_month)

```

```

df["target_month"] = target_month

rows.append(df.reset_index())

features = pd.concat(rows, ignore_index=True)
features = features.sort_values("target_month", kind="mergesort").reset_index(drop=True)

assert features["target_month"].is_monotonic_increasing
assert (features["target_month"].value_counts() == GROUPS).all()

return features

# Baseline #2: last 3 months average

def make_last_3_avg_features(group_ts):
    rows = []

    for target_month in TARGET_FULL_RANGE:
        start = target_month - pd.offsets.MonthBegin(3)
        end = target_month - pd.offsets.MonthBegin(1)

        cols = pd.date_range(start, end, freq="MS")
        cols = [c for c in cols if c in group_ts.columns]

        if cols:
            avg = group_ts.loc[:, cols].mean(axis=1)
        else:
            avg = pd.Series(0.0, index=group_ts.index)

        df = pd.DataFrame(index=group_ts.index)
        df["last_3m_avg"] = avg
        df["target"] = month_value(group_ts, target_month)
        df["target_month"] = target_month

        rows.append(df.reset_index())

    features = pd.concat(rows, ignore_index=True)
    features = features.sort_values("target_month", kind="mergesort").reset_index(drop=True)

    assert features["target_month"].is_monotonic_increasing
    assert (features["target_month"].value_counts() == GROUPS).all()

    return features

# Baseline #3: CatBoostRegressor

def get_features(group_ts: pd.DataFrame, month: pd.Timestamp) -> pd.DataFrame:
    """Признаки для месяца `month`. В точности как в ноутбукe."""
    start_period = month - pd.offsets.MonthBegin(6)
    end_period = month - pd.offsets.MonthBegin(1)

    df = group_ts.loc[:, :end_period]

    features = pd.DataFrame([], index=df.index)
    features["month"] = month.month
    features[[f"vol_tm{i}" for i in range(6, 0, -1)]] = df.loc[:, start_period:end_period].copy()

    roll_t = df.T.rolling(window=12, min_periods=1)

```

```

features = features.join(roll_t.mean().iloc[-1].rename("last_year_avg"))
features = features.join(roll_t.min().iloc[-1].rename("last_year_min"))
features = features.join(roll_t.max().iloc[-1].rename("last_year_max"))
return features

def make_catboost_features(group_ts):
    datasets = []
    for target_month in TARGET_FULL_RANGE:
        features = get_features(group_ts, target_month)
        features["target"] = month_value(group_ts, target_month)
        features["target_month"] = target_month
        datasets.append(features.reset_index())

    df = pd.concat(datasets, ignore_index=True)
    df = df.sort_values("target_month", kind="mergesort").reset_index(drop=True)

    assert df["target_month"].is_monotonic_increasing
    assert (df["target_month"].value_counts() == GROUPS).all()

    return df

def evaluate_catboost_baseline(df, name):
    print(f"\n=== {name} ===")

    base_model = CatBoostRegressor(
        iterations=1000,
        early_stopping_rounds=30,
        depth=6,
        cat_features=CAT_COLS,
        random_state=RS,
        verbose=False,
        allow_writing_files=False,
    )

    ts_cv = TimeSeriesSplit(n_splits=5, test_size=GROUPS * 3)
    losses = []

    for fold, (train_idx, test_idx) in enumerate(ts_cv.split(df), 1):
        X_train = df[FTS_COLS].iloc[train_idx]
        X_test = df[FTS_COLS].iloc[test_idx]
        y_train = df[TARGET].iloc[train_idx]
        y_test = df[TARGET].iloc[test_idx]

        model = base_model.copy()
        model.fit(X_train, y_train, eval_set=(X_test, y_test))

        train_loss = rmsle(y_train, model.predict(X_train))
        test_loss = rmsle(y_test, model.predict(X_test))

        losses.append([train_loss, test_loss])

    print(f"Fold {fold}")
    print(
        f" Train: {df['target_month'].iloc[train_idx].min().date()} - "
        f"{df['target_month'].iloc[train_idx].max().date()} | RMSLE={train_loss:.6f}"
    )
    print(
        f" Test: {df['target_month'].iloc[test_idx].min().date()} - "

```

```

        f"{df['target_month'].iloc[test_idx].max().date()} | RMSLE={test_loss:.6f}"
    )

    avg = np.mean(losses, axis=0)
    print(f"Avg Train RMSLE: {avg[0]:.6f}, Avg Test RMSLE: {avg[1]:.6f}")

def main():
    group_ts = load_group_ts()
    print(f"Group time series shape: {group_ts.shape}")

    last_value = make_last_value_features(group_ts)
    evaluate_simple(last_value, "vol_tm1", "Baseline #1: last value")

    last_3_avg = make_last_3_avg_features(group_ts)
    evaluate_simple(last_3_avg, "last_3m_avg", "Baseline #2: last 3 months average")

    catboost_features = make_catboost_features(group_ts)
    evaluate_catboost_baseline(catboost_features, "Baseline #3: CatBoostRegressor")

if __name__ == "__main__":
    main()

```

main.py

```

import numpy as np
import pandas as pd
from sklearn.model_selection import TimeSeriesSplit
from catboost import CatBoostRegressor

from common import (
    AGG_COLS,
    GROUPS,
    TARGET_FULL_RANGE,
    RS,
    rmsle,
    load_group_ts,
    month_value,
)

CAT_COLS = AGG_COLS + ["month"]
TARGET = "target"

def get_features_for_month(group_ts, target_month):
    """Признаки для одного target_month. Используется только история до target_month."""
    end = target_month - pd.offsets.MonthBegin(1)
    hist = group_ts.loc[:, :end]

    if hist.shape[1] == 0:
        hist = pd.DataFrame(0.0, index=group_ts.index, columns=[end])

    features = pd.DataFrame(index=group_ts.index)

    # Календарные признаки
    features["month"] = target_month.month
    features["quarter"] = target_month.quarter
    features["year"] = target_month.year
    features["month_idx"] = (target_month.year - 2018) * 12 + target_month.month

```

```

features["month_sin"] = np.sin(2 * np.pi * target_month.month / 12)
features["month_cos"] = np.cos(2 * np.pi * target_month.month / 12)

```

```

# Лаги 1..12

```

```

for lag in range(1, 13):
    m = target_month - pd.offsets.MonthBegin(lag)
    features[f"vol_tm{lag}"] = month_value(group_ts, m)

```

```

# Скользящие статистики (по времени, т.е. по столбцам hist)

```

```

# Транспонируем, чтобы rolling работал вдоль оси времени

```

```

hist_t = hist.T # shape: (месяцы, группы)

```

```

for w in (3, 6, 12):
    roll_t = hist_t.rolling(window=w, min_periods=1)
    features[f"mean_{w}"] = roll_t.mean().iloc[-1]
    features[f"std_{w}"] = roll_t.std().iloc[-1].fillna(0.0)
    features[f"min_{w}"] = roll_t.min().iloc[-1]
    features[f"max_{w}"] = roll_t.max().iloc[-1]

```

```

# Статистики по всей истории

```

```

features["hist_mean"] = hist.mean(axis=1)
features["hist_std"] = hist.std(axis=1).fillna(0.0)
features["hist_min"] = hist.min(axis=1)
features["hist_max"] = hist.max(axis=1)
features["hist_nonzero_ratio"] = (hist > 0).mean(axis=1)

```

```

# Тренды и отношения

```

```

features["diff_1_2"] = features["vol_tm1"] - features["vol_tm2"]
features["ratio_1_2"] = features["vol_tm1"] / (features["vol_tm2"] + 1.0)
features["diff_1_3"] = features["vol_tm1"] - features["vol_tm3"]
features["diff_1_6"] = features["vol_tm1"] - features["mean_6"]
features["diff_1_12"] = features["vol_tm1"] - features["mean_12"]

```

```

features = features.fillna(0.0)

```

```

# Целевая переменная

```

```

features[TARGET] = month_value(group_ts, target_month)

```

```

return features

```

```

def build_dataset(group_ts):

```

```

    rows = []

```

```

    for target_month in TARGET_FULL_RANGE:

```

```

        df = get_features_for_month(group_ts, target_month).reset_index()

```

```

        df["target_month"] = target_month

```

```

        rows.append(df)

```

```

    df = pd.concat(rows, ignore_index=True)

```

```

    df = df.sort_values("target_month", kind="mergesort").reset_index(drop=True)

```

```

    assert df["target_month"].is_monotonic_increasing

```

```

    assert (df["target_month"].value_counts() == GROUPS).all()

```

```

    return df

```

```

def run_cv(df):
    exclude = {"target", "target_month"}
    feature_cols = [c for c in df.columns if c not in exclude]

    ts_cv = TimeSeriesSplit(n_splits=5, test_size=GROUPS * 3)
    losses = []

    for fold, (train_idx, test_idx) in enumerate(ts_cv.split(df), 1):
        train = df.iloc[train_idx]
        test = df.iloc[test_idx]

        X_train = train[feature_cols]
        X_test = test[feature_cols]

        y_train = train[TARGET].values
        y_test = test[TARGET].values

        model = CatBoostRegressor(
            iterations=3000,
            learning_rate=0.03,
            depth=6,
            l2_leaf_reg=3.0,
            loss_function="RMSE",
            eval_metric="RMSE",
            random_seed=RS,
            od_type="Iter",
            od_wait=100,
            verbose=False,
            cat_features=CAT_COLS,
            allow_writing_files=False,
        )

        # Важно: учим на log1p(target), т.к. RMSLE = RMSE на log1p
        model.fit(
            X_train,
            np.log1p(y_train),
            eval_set=(X_test, np.log1p(y_test)),
            use_best_model=True,
        )

        train_pred = np.expm1(model.predict(X_train))
        test_pred = np.expm1(model.predict(X_test))

        train_loss = rmsle(y_train, train_pred)
        test_loss = rmsle(y_test, test_pred)

        losses.append([train_loss, test_loss])

    print(f"Fold {fold}")
    print(
        f" Train: {train['target_month'].min().date()} - "
        f"{train['target_month'].max().date()} | RMSLE={train_loss:.6f}"
    )
    print(
        f" Test: {test['target_month'].min().date()} - "
        f"{test['target_month'].max().date()} | RMSLE={test_loss:.6f}"
    )

    avg = np.mean(losses, axis=0)

```

```
print(f"\nAvg Train RMSLE: {avg[0]:.6f}, Avg Test RMSLE: {avg[1]:.6f}")

def main():
    group_ts = load_group_ts()
    print(f"Group time series shape: {group_ts.shape}")

    df = build_dataset(group_ts)
    print(f"CatBoost dataset shape: {df.shape}")

    run_cv(df)

if __name__ == "__main__":
    main()
```